

HONEYBADGER SCHOOL — DATABASE MIGRATION ROADMAP

Supabase Backend Schema & Migration Step-by-Step Guide

■ GREEN = Table Name

■ YELLOW = Columns

■ RED = Row Logic & Rules

STEP 1: SYSTEM CORE & COUNTERS

■ schools

id, name, name_am, city, phone, logo_url, timezone, created_at, deleted_at

Rules: Tenant root row; default timezone 'Africa/Addis_Ababa'; readable by staff, writable by platform admin only.

■ school_counters

school_id (PK), next_server_seq, next_receipt_block

Rules: 1 sequence allocator row per school; updated inside transaction via UPDATE...RETURNING; no direct client access.

STEP 2: ACADEMIC CALENDAR

■ academic_years

id, school_id, label, starts_on, ends_on, is_current, created_at

Rules: Unique (school_id, label); partial unique on is_current stops multiple active years; check (ends_on > starts_on).

■ terms

id, school_id, academic_year_id, name, seq, starts_on, ends_on, is_current, closed_at

Rules: Unique (academic_year_id, seq); setting closed_at permanently freezes grading weights for that term.

STEP 3: IDENTITY & ROLE GOVERNANCE

■ users

id (→ auth.users.id), school_id, first_name, father_name, sex, position, employment_type, hire_date, phone, email, status, departed_on, server_seq, deleted_at, created_at

Rules: References auth.users; never hard deleted (retire via status); phone/email columns are restricted to self & admins.

■ user_roles

user_id, role, granted_at, granted_by

Rules: PK (user_id, role); multi-role support per staff member (admin, teacher, registrar, cashier); parsed in JWT claim.

STEP 4: GUARDIANS & CONTACTS

■ guardians

id, school_id, full_name, phone, alt_phone, email, national_id, auth_user_id, server_seq, deleted_at, created_at

Rules: Non-unique index (school_id, phone) allows shared numbers; auth_user_id set when parent portal account is activated.

■ student_guardians

student_id, guardian_id, relationship, is_primary, can_collect, created_at

Rules: PK (student_id, guardian_id); partial unique constraint guarantees exactly 1 primary contact row per student.

STEP 5: ACADEMIC HIERARCHY

■ grade_levels

id, school_id, level, name

Rules: Unique (school_id, level); defines academic grade tiers (e.g. Grade 1 to 12).

■ classes

id, school_id, grade_level_id, name, deleted_at

Rules: Unique (school_id, grade_level_id, name); defines stable class streams across years (e.g. "Section A").

■ subjects

id, school_id, name, code, deleted_at

Rules: Unique (school_id, name); master list of curriculum subjects across the institution.

STEP 6: YEARLY ASSIGNMENTS

■ class_years

id, school_id, class_id, academic_year_id, homeroom_user_id, capacity

Rules: Unique (class_id, academic_year_id); preserves historical homeroom teacher assignments per academic year.

■ teaching_assignments

id, school_id, user_id, class_id, subject_id, academic_year_id

Rules: Unique (user_id, class_id, subject_id, academic_year_id); grants subject-level write RLS access to assigned teachers.

STEP 7: ROSTER & STUDENT LIFECYCLES

■ students

id, school_id, student_no, school_ref_no, first_name, father_name, grandfather_name, sex, date_of_birth, joined_on, status, server_seq, deleted_at, created_at

Rules: Unique (school_id, student_no); student lifecycle status (active, graduated, left); contains no direct class/grade columns.

■ enrollments

id, school_id, student_id, academic_year_id, class_id, status, outcome, enrolled_on, ended_on, server_seq, created_at

Rules: Unique (student_id, academic_year_id); yearly grade placement; promotions insert new enrollment rows without overwriting history.

STEP 8: DAILY ATTENDANCE OPERATIONS

■ attendance_registers

id, school_id, class_id, business_date, submitted_at, submitted_by, server_seq, created_at

Rules: Unique (class_id, business_date); register submission operates as a monotonic latch (false -> true only).

■ attendance_records

id, school_id, class_id, student_id, business_date, mark, marked_by, client_recorded_at, server_seq, deleted_at

Rules: Unique (student_id, business_date); per-student attendance patch sync; mark options = P, A, L.

■ teacher_attendance

id, school_id, user_id, business_date, checked_in_at, checked_out_at, source, status, approved_by, approved_at, client_recorded_at, server_seq

Rules: Unique (user_id, business_date); flags server_stamped vs device_unverified to manage offline check-in validation.

STEP 9: ASSESSMENT STRUCTURE

■ assessment_types

key, name, seq, id, school_id

Rules: Unique (school_id, key); standardized grading category labels across the school.

■ subject_weights

id, school_id, academic_year_id, term_id, grade_level_id, subject_id, assessment_type_id, weight, locked_at

Rules: Unique (term_id, grade_level_id, subject_id, assessment_type_id); total weights per subject must equal 100 via trigger.

STEP 10: EXAMINATION & SCORES

■ assessments

id, school_id, class_id, subject_id, assessment_type_id, term_id, title, max_score, business_date, created_by, client_recorded_at, server_seq, deleted_at

Rules: Enforces max_score > 0; indexed on (school_id, class_id, subject_id, term_id) for fast query speeds.

■ assessment_scores

id, school_id, assessment_id, student_id, raw_score, entered_by, client_recorded_at, server_seq, deleted_at

Rules: Unique (assessment_id, student_id); stores raw marks only; NULL represents work not sat (distinct from score 0).

■ assessment_score_history

id, school_id, assessment_id, student_id, old_score, new_score, changed_by, reason, source, server_seq, at

Rules: Automated trigger log capturing every mark revision and offline sync conflict resolution loss.

■ report_comments

id, school_id, student_id, term_id, comment, updated_by, server_seq

Rules: Unique (student_id, term_id); homeroom teacher narrative commentary per term.

STEP 11: FEE SCHEDULES

■ fee_items

id, school_id, academic_year_id, grade_level_id, name, amount_santim, kind, term_id, deleted_at

Rules: Enforces amount_santim >= 0 (1 ETB = 100 santim); fee structures defined per academic year and grade level.

STEP 12: PAYMENTS & POS LEASING

■ payments

id, school_id, student_id, receipt_no, total_santim, method, received_by, business_date, client_recorded_at, server_seq, op_id, voided_at, voided_by, void_reason

Rules: Unique (school_id, receipt_no) & unique (op_id); append-only transaction ledger; errors must be corrected by voiding.

■ payment_lines

id, school_id, payment_id, fee_item_id, label, amount_santim

Rules: Enforces amount_santim > 0; label is denormalized to freeze text on historical printed receipts.

■ receipt_leases

id, school_id, device_id, block_start, block_end, next_number, leased_at, exhausted_at

Rules: Unique (school_id, block_start); pre-allocates receipt ranges to prevent numbering collisions between offline mobile POS units.

STEP 13: SYNC & OFFLINE QUEUES

■ change_log

seq, school_id, table_name, row_id, op, payload, actor_user_id, server_time

Rules: PK (school_id, seq); append-only delta stream generated automatically by unified system triggers.

■ sync_devices

id, school_id, device_id, user_id, cursor_seq, scope_version, platform, last_seen_at, app_version

Rules: Unique (school_id, device_id); tracks device cursors and triggers history backfills when user role scope changes.

■ processed_ops

op_id (PK), school_id, device_id, table_name, row_id, result, server_seq, processed_at

Rules: Idempotency tracking table retaining operation records (~30 days) to eliminate duplicate mutations during network retry attempts.

STEP 14: SYSTEM AUDIT TRAIL

■ audit_log

id, school_id, actor_user_id, subject_table, subject_id, action, before, after, server_seq, at

Rules: Strictly ordered by server_seq; immutable log of administrative edits and security modifications.